
EKBA — COMPLETE PROJECT FLOW

Same Master Flow for Notebook / Revision

Ye document exactly wahi complete flow hai jo abhi explain kiya gaya tha, notebook me likhne aur revise karne ke liye.

1. ■ USER → FRONTEND

User
↓
Next.js + React
↓
AWS Amplify

2. ■ AUTHENTICATION

Login
↓
Amazon Cognito
↓
ID Token / JWT
↓
custom:tenant_id + custom:role
↓
FastAPI JWT Verification

Roles:

Platform Admin
↓
Company
↓
Company Admin
↓
Normal Users

3. ■ API REQUEST FLOW

User Browser
↓
Amplify
↓
CloudFront
↓
ALB
↓
ECS Fargate
↓
FastAPI

FastAPI = project ka main backend/orchestrator

4. ■ DOCUMENT UPLOAD FLOW

User uploads PDF
↓
POST /api/v1/documents
↓
JWT Verify
↓
Tenant Verify
↓
S3
↓
PostgreSQL
■■■ documents
■■■ ingestion_jobs
↓
Background Ingestion
↓
Text Extraction
↓
Chunking
↓
Embedding
↓
Qdrant

Extraction

PDF
↓
pypdf
↓
Extracted Text

Dedicated Textract/Tesseract OCR **implemented nahi hai**.

5. ✂️ CHUNKING

Extracted Text
↓
Chunker
↓
512 Tokens
↓
64 Tokens Overlap
↓
Small Knowledge Chunks

Overlap ka purpose → chunks ke beech context continuity maintain karna.

6. ■ EMBEDDING

Active Model:

Cohere
↓
embed-multilingual-v3.0
↓
1024 Dimensions

Text Chunk
↓
Embedding Model
↓
[0.12, -0.45, 0.78,]
1024 values

Then:

1024-D Vector
↓
Qdrant Cloud

Qdrant:

Collection = ekba_chunks
Distance = COSINE

7. ■■ DATA STORAGE

Service	Kya store hota hai
Cognito	Users + authentication
PostgreSQL/RDS	Tenant, users, documents, jobs, chat, usage, audit
S3	Original PDFs/files
Qdrant Cloud	Embeddings + chunks + metadata
Redis	Cache + rate limiting
Secrets Manager	API keys + DB credentials
CloudWatch	Logs + alarms

8. ■ USER QUESTION / RAG FLOW

Example:

"What is the daily meal allowance?"

User Question ↓ POST /api/v1/chat ↓ JWT Verify ↓ Question Validation ↓ Prompt Injection Scan ↓ Cohere
Embedding ↓ Redis Semantic Cache ↓ Qdrant Search ↓ Tenant ID Filter ↓ Similarity Threshold ↓ BM25 +
Vector Reranking ↓ Top 8 Chunks ↓ Context + System Prompt ↓ Groq LLM ↓ Citation Validation ↓ Output
Guardrail ↓ Answer + Citations ↓ Frontend

9. ■ ACTIVE LLM

Current active: Provider = Groq Model = openai/gpt-oss-20b

LLM ko direct question ke saath company ka complete database nahi diya jaata. Instead: Question + Relevant Qdrant Chunks + System Prompt ↓ Groq LLM ↓ Answer

Yahi RAG hai.

10. ■ QDRANT KA ROLE

Qdrant me approximately: Vector + Chunk Text + Metadata

Example: tenant_id document_id document_name page_number source_uri text department suspicious

Most important: User Tenant ↓ tenant_id filter ↓ Qdrant Search

Isse Tenant A ko Tenant B ka document retrieve nahi hoga.

11. ■ SECURITY FLOW

Cognito JWT ↓ Authentication ↓ RBAC ↓ Tenant Isolation ↓ SQL Tenant Filter ↓ S3 Tenant Prefix ↓ Qdrant tenant_id Filter ↓ Redis Tenant Namespace

Additional: Secrets Manager Prompt Injection Scan Output Guardrails Citation Validation Rate Limiting
File Validation Security Groups HTTPS / CloudFront GitHub OIDC

12. ■■ AWS DEPLOYMENT FLOW

Developer ↓ git push ↓ GitHub ↓ GitHub Actions ↓ OIDC Authentication ↓ Docker Build ↓ Amazon ECR ↓ New
ECS Task Definition ↓ AWS CodeDeploy ↓ GREEN Environment ↓ Health Check ↓ Traffic Shift ↓ BLUE → OLD
GREEN → PRODUCTION

Blue-Green:

ALB | ■■■■■■■■■■■■■■■■■■■■■■ ↓ ↓ BLUE TG GREEN TG Current New ↓ Health Check ↓ Traffic Shift

13. ■ MONITORING

ECS / FastAPI ↓ CloudWatch Logs ↓ Alarms ■■■ 5xx ■■■ Latency

5xx alarm deployment rollback behavior ko support karta hai.

14. ■ SECRETS

AWS Secrets Manager ↓ IAM Execution Role ↓ ECS Task ↓ Environment / Pydantic Settings

Examples: COHERE_API_KEY GROQ_API_KEY QDRANT_API_KEY DATABASE_URL

Secrets GitHub/frontend me nahi rakhne.

15. ■ COST-SAVING DESTROY FLOW

Jab demo/interview nahi hai: `./scripts/destroy.sh` ↓ RDS Snapshot ↓ Terraform Destroy ↓ Ephemeral AWS Resources Deleted

Protected resources survive:

Cognito ECR S3 Secrets Manager Terraform State Qdrant Cloud Frontend RDS Snapshot

Phir interview/demo ke time:

`./scripts/deploy.sh` ↓ Recreate Infrastructure ↓ Restore RDS Snapshot ↓ ECS + CodeDeploy ↓
`./scripts/deploy-frontend.sh`

■ FINAL MASTER FLOW

```
USER ↓ Next.js / React ↓ Amplify ↓ CloudFront ↓ ALB ↓ ECS Fargate ↓ FastAPI ↓ Cognito JWT ↓  
██████████████████████████████ ↓ ↓ Tenant + RBAC API Logic ↓  
██████████████████████████████ ↓ ↓ ↓ S3 PostgreSQL Redis Qdrant Original Files  
Business DB Cache Vectors ↓ Tenant Filter ↓ Relevant Chunks ↓ Cohere Embedding ↓ Groq LLM  
openai/gpt-oss-20b ↓ Citation Validation ↓ Output Guardrail ↓ Answer + Citations ↓ USER
```

■ ONE SENTENCE TO REMEMBER

“User Cognito se authenticate hota hai, request CloudFront → ALB → ECS FastAPI tak jaati hai; documents S3 me aur metadata Postgres me store hote hain, chunks Cohere embeddings ke through Qdrant me jaate hain, user question par tenant-filtered Qdrant retrieval hoti hai, relevant context Groq **openai/gpt-oss-20b** ko diya jaata hai, phir citations aur guardrails ke baad answer user ko milta hai.”